



□ 特征处理（半结构化→结构化）

- 统一度量衡：如租期（包含“月/年”单位）、梯户比等
- One-hot处理：租赁方式、建筑结构、供暖方式等
- 文本数据取字符串长度：房屋优势、核心卖点等
- 调用deepseek大语言模型：客户反馈特征
- 缺失值+异常值处理

□ SHAP+PDP——特征选择（为未来改进方向做准备）

- 发现迁移学习变量重要性最高且特征很干净即价格越高租金越高——房价预测特征可以为租金预测提供额外的信息
- 发现调用大模型获得的“比较好”等评价对模型有一定预测效果——但重要性排序没那么高，后面优先模型的优化
- 地理位置、房屋时间和房屋基本情况是影响最大的三大要素

□ 因变量变化：房价是严重右偏、长尾分布、异方差的

- 尝试1：取ln
- 尝试2：取box-cox变换——ln是其一种特殊情况，可以更好解决房价问题

$$y^{(\lambda)} = \begin{cases} \frac{y^\lambda - 1}{\lambda}, & \lambda \neq 0 \\ \ln(y), & \lambda = 0 \end{cases}$$

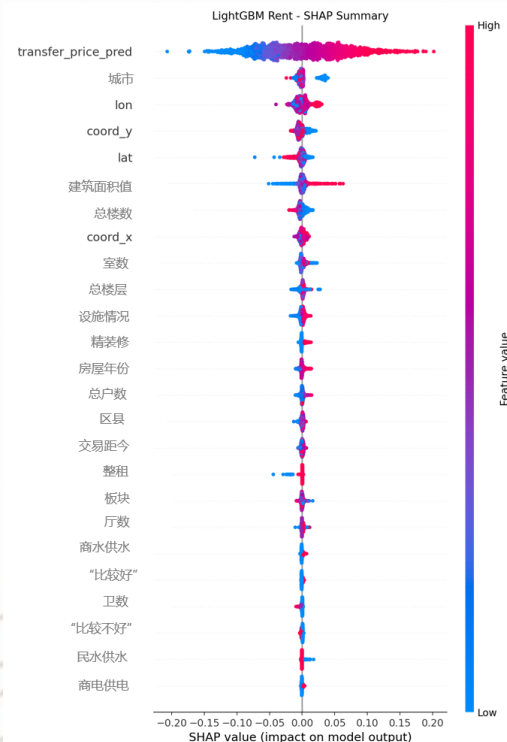
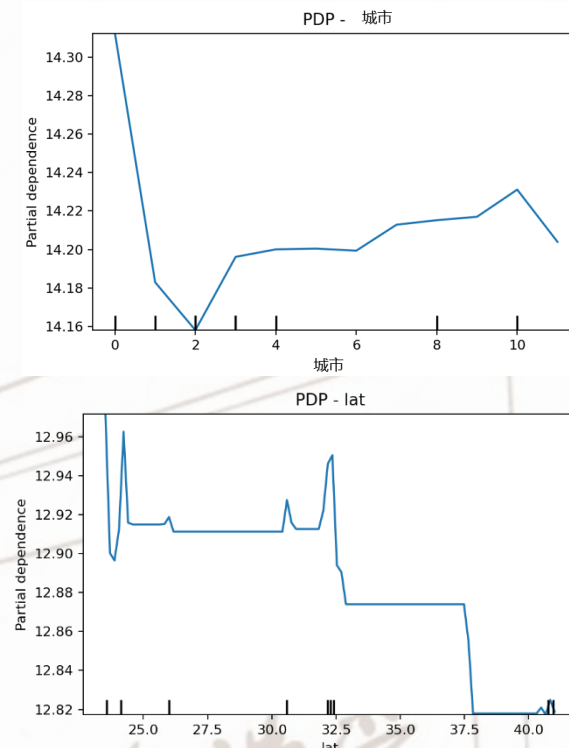


图1 租金预测LGBM模型的SHAP



Model	Price in rmse	Price oos rmse	Price cv rmse	kaggle
Ridge [noT]+boxcox	0.127297	0.128136	0.127568	37.1
Ridge [noT]+ln	0.481844	0.484590	0.482788	36.2
Ridge [T]+boxcox	0.068723	0.069805	0.068995	65.7
Ridge [T]+ln	0.258676	0.261852	0.259523	66.2
XGBoost [noT]+boxcox	0.009964	0.031141	0.030941	78.0
XGBoost [noT]+ln	0.036813	0.116580	0.115948	77.9
XGBoost [T]+boxcox	0.009748	0.030971	0.030775	77.7
XGBoost [T]+ln	0.036500	0.115985	0.115063	77.6



□ 多种模型实验

- 采用了线性模型Ridge、XGboost、 RandomForest、 Catboost、 LightGBM、 ANN等多种模型
- 其中XGboost效果最好

□ 迁移模型

- S1. 找到房价与租金预测的共同特征
- S2. 利用LightGBM模型拟合迁移模型
- S3. 利用房价特征预测rent数值，并将预测值视为房价预测的一个特征
- 最终迁移模型对线性模型有较好作用，对部分树模型有效，但对拟合效果本身较高的模型则有负向作用

□ Optuna调参

- 把 LGBM 的关键超参数纳入 trial 搜索，并用 CV RMSE + early stopping 作为目标函数
- 优化整体运行性能

表1 部分模型结果展示

Model	Price_in_rmse	Price_oos_rmse	Price_cv_rmse	kaggle
LightGBM tuned [noT]+boxcox	0.011830	0.032324	0.032133	0.012267
LightGBM tuned [noT]+ln	0.048039	0.120629	0.119554	0.103986
LightGBM tuned [T]+boxcox	0.010891	0.032339	0.031936	0.011685
LightGBM tuned [T]+ln	0.046145	0.119680	0.118691	0.099884
XGBoost [noT]+boxcox	0.009964	0.031141	0.030941	78.0
XGBoost [noT]+ln	0.036813	0.116580	0.115948	77.9
XGBoost [T]+boxcox	0.009748	0.030971	0.030775	77.7
XGBoost [T]+ln	0.036500	0.115985	0.115063	77.6
cv_weighted [noT]+ln	/	/	/	77.8
cv_weighted [T]+ln	/	/	/	77.7
cv_weighted [noT]+boxcox	/	/	/	77.9
cv_weighted [T]+boxcox	/	/	/	77.7

□ 多模型加权集成：用 CV 表现反推权重

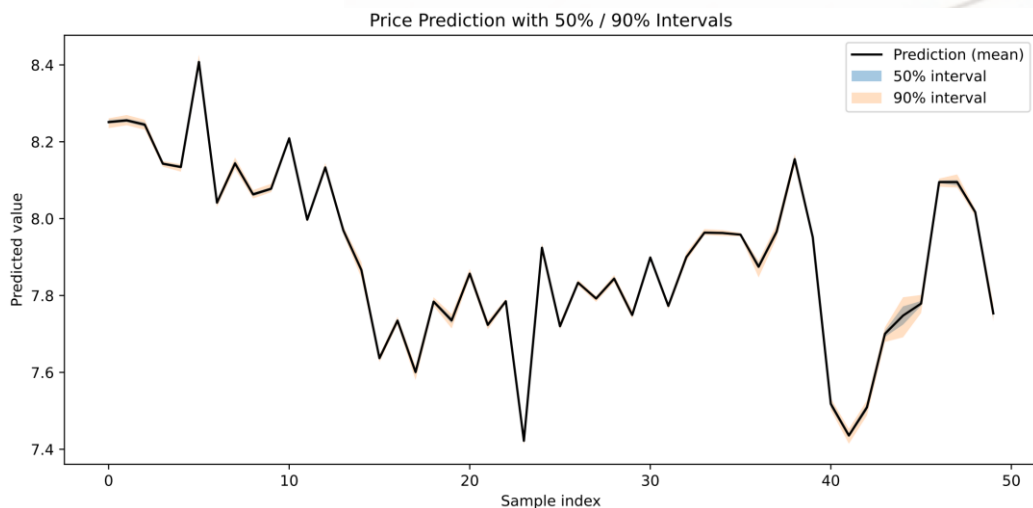
- 谁的 CV 表现更好，谁权重更高
- 采用所有模型进行集成
- 该系列整体分数都较高，模型较为稳定

□ 全流程 Pipeline 化

- 训练/验证/预测统一接口, 降低重复与出错率
- 预处理 → (可选标准化) → 模型 → split 验证 → CV
→ final fit → test pred 全部统一起来

□ 区间预测 (Prediction Interval) :

- 一般预测中会给出区间预测结果, 以便进行真实情况预估
- Bootstrap RF 给出 50%/90% 区间
- 对训练集做有放回抽样, 训练多次 RF, 得到测试集预测分布, 然后取分位数形成区间 (p05/p25/p75/p95 等)



```
def build_preprocessor(X: pd.DataFrame):  
    """  
    统一预处理: 缺失值处理 + 类别OneHot  
    【关键点】统一Pipeline, 避免数据泄漏  
    """  
    numeric_cols = X.select_dtypes(include=[np.number]).columns.tolist()  
    categorical_cols = [c for c in X.columns if c not in numeric_cols]  
  
    numeric_tf = Pipeline(steps=[  
        ("imputer", SimpleImputer(strategy="median")),  
    ])  
    categorical_tf = Pipeline(steps=[  
        ("imputer", SimpleImputer(strategy="most_frequent")),  
        ("onehot", OneHotEncoder(handle_unknown="ignore")),  
    ])  
  
    preprocessor = ColumnTransformer(  
        transformers=[  
            ("num", numeric_tf, numeric_cols),  
            ("cat", categorical_tf, categorical_cols),  
        ],  
        remainder="drop"  
    )  
    return preprocessor, numeric_cols, categorical_cols
```

